

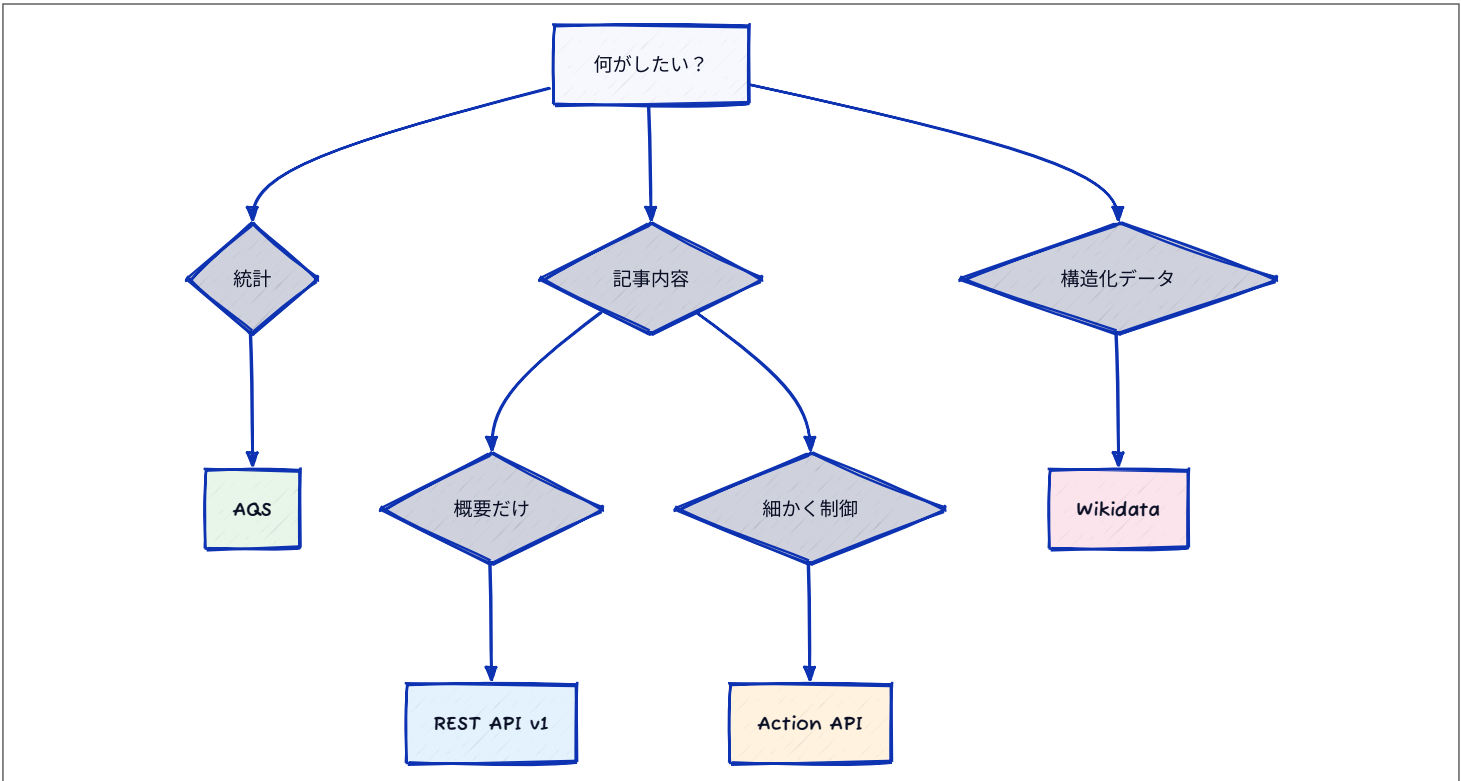
このスライドの位置づけ

- これは **リファレンス（参照資料）**
 - 全部覚える必要はない。**必要なときに引く**
- 前提知識は **APIでデータ取得** [🔗](#)（requests、エンドポイント、User-Agent）

💡 使い方

1. 「こういうデータ欲しいな」と思う
2. このスライドで **どのAPIか** を探す
3. エンドポイントとパラメータをコピーして使う

どのAPIを使うべきか



色	API	用途	例
緑	AQS	ページビュー統計	APIでデータ取得 🔗 で使用
青	REST API v1	概要をサクッと	/page/summary/桜
橙	Action API	検索・カテゴリ・履歴	action=query&list=search
桃	Wikidata	機械可読な構造化データ	entities/items/Q1490

Wikipedia API の全体像

3つのAPIファミリー

API	特徴	用途
Action API	豊富なクエリ・編集・コンテンツアクセス	検索、ページ情報、履歴取得
REST API v1	高負荷コンテンツアクセスに特化	概要取得、HTML/PDF変換
Analytics Query Service	統計データとメトリクス	ページビュー、編集統計

基本情報

項目	内容
認証	基本的に不要（一部機能は要認証）
レート制限	200リクエスト/秒（全体）
データ形式	JSON / XML / HTML
CORS	サポート済み

💡 授業での利用

- 読み取り専用なら認証不要
- User-Agent** ヘッダーは必須

6

Action API

7

Action API：基本URL

```
1 https://ja.wikipedia.org/w/api.php
```

- パラメータはクエリ文字列（**?action=...&prop=...**）で渡す
- 例：「桜」で記事を検索して上位5件を取得

```
1 https://ja.wikipedia.org/w/api.php?action=query&list=search&srsearch=桜&format=json&srlimit=5
```

8

検索系

エンドポイント	説明	パラメータ例
<code>action=query&list=search</code>	記事検索	<code>srsearch=桜&srlimit=10</code>
<code>action=opensearch</code>	オートコンプリート	<code>search=桜&limit=8</code>
<code>action=query&list=random</code>	ランダムページ	<code>rnlimit=10&rnnamespace=0</code>

ページ情報

エンドポイント	説明	パラメータ例
<code>action=query&prop=info</code>	基本情報	<code>titles=桜&prop=info</code>
<code>action=query&prop=extracts</code>	要約テキスト	<code>titles=桜&exintro&explaintext</code>
<code>action=query&prop=images</code>	画像一覧	<code>titles=桜&prop=images</code>
<code>action=query&prop=categories</code>	カテゴリ	<code>titles=桜&prop=categories</code>
<code>action=query&prop=pageviews</code>	過去60日のPV	<code>titles=桜&prop=pageviews</code>

リンク・履歴

エンドポイント	説明	パラメータ例
<code>action=query&prop=links</code>	ページ内リンク	<code>titles=桜&prop=links</code>
<code>action=query&list=backlinks</code>	被リンク一覧	<code>bltitle=桜&bllimit=50</code>
<code>action=query&prop=revisions</code>	編集履歴	<code>titles=桜&prop=revisions&rvlimit=10</code>
<code>action=query&list=recentchanges</code>	最近の編集集	<code>rclimit=50&rcnamespace=0</code>

カテゴリ・ユーザー

エンドポイント	説明	パラメータ例
<code>action=query&list=categorymembers</code>	カテゴリ内ページ	<code>cmtitle=Category:桜&cmlimit=50</code>
<code>action=query&list=usercontribs</code>	ユーザー編集履歴	<code>ucuser=UserName&uclimit=50</code>

REST API v1：基本URL

1 https://ja.wikipedia.org/api/rest_v1/

- **パスにパラメータを埋め込む** タイプ
- 例：「桜」の記事概要を取得

1 https://ja.wikipedia.org/api/rest_v1/page/summary/桜

REST API v1：エンドポイント

ページ系

エンドポイント	説明	形式
/page/summary/{title}	ページ概要	JSON
/page/html/{title}	HTMLコンテンツ	HTML
/page/mobile-html/{title}	モバイル用HTML	HTML
/page/media-list/{title}	メディアファイル一覧	JSON
/page/pdf/{title}	PDF生成	PDF
/page/random/summary	ランダム概要	JSON

変換・フィード

エンドポイント	説明	形式
/transform/wikitext/to/html	Wikitext→HTML	HTML
/transform/html/to/wikitext	HTML→Wikitext	Text
/feed/featured/{year}/{month}/{day}	注目記事	JSON
/feed/onthisday/{type}/{month}/{day}	今日は何の日（日本語版は未対応）	JSON

Analytics Query Service（AQS）

AQS：基本URL

1 https://wikimedia.org/api/rest_v1/metrics/

- **APIでデータ取得** [🔗](#)で使ったのはこれ
- 例：「桜」の2024年3月の日別アクセス数

1 https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article/ja.wikipedia.org/all-access/all-agents/桜/daily/20240301

AQS：エンドポイント

ページビュー

per-article — 記事別PV（2015年7月～）

```
1 pageviews/per-article/{project}/{access}/{agent}/{article}/{granularity}/{start}/{end}
```

例：pageviews/per-article/ja.wikipedia.org/all-access/all-agents/東京/daily/20240101/20240131

aggregate — プロジェクト全体PV（2015年7月～）

```
1 pageviews/aggregate/{project}/{access}/{agent}/{granularity}/{start}/{end}
```

例：pageviews/aggregate/ja.wikipedia.org/all-access/all-agents/daily/2024010100/2024013100（aggregate だけ日付は **YYYYMMDDHH** の10桁）

top — 人気記事ランキング（2015年7月～）

```
1 pageviews/top/{project}/{access}/{year}/{month}/{day}
```

例：pageviews/top/ja.wikipedia.org/all-access/2024/06/01

その他統計

unique-devices — ユニークデバイス数（2016年1月～。ただしデータ欠落期間があり、範囲に欠落を含むと404になる）

```
1 unique-devices/{project}/{access-site}/{granularity}/{start}/{end}
```

例：unique-devices/ja.wikipedia.org/all-sites/daily/20250101/20250131

edits — 編集統計（2001年1月～）

```
1 edits/aggregate/{project}/{editor-type}/{page-type}/{granularity}/{start}/{end}
```

例：edits/aggregate/ja.wikipedia.org/all-editor-types/all-page-types/daily/20240101/20240131

registered-users — 新規ユーザー登録（2001年1月～）

```
1 registered-users/new/{project}/{granularity}/{start}/{end}
```

例：registered-users/new/ja.wikipedia.org/daily/20240101/20240131

15

Wikidata REST API

16

Wikidata API：基本URL

```
1 https://www.wikidata.org/w/rest.php/wikibase/v1/
```

- Wikipedia記事には内部ID（**Wikidata ID**）がある
- 例：東京都（Q1490）のアイテム情報

```
1 https://www.wikidata.org/w/rest.php/wikibase/v1/entities/items/Q1490
```

17

Wikidata API：エンドポイント

エンドポイント	説明	操作
entities/items/{item_id}	アイテム取得	Read
entities/items	アイテム作成	Create
entities/properties/{property_id}	プロパティ取得	Read
entities/items/{item_id}/statements	ステートメント一覧	Read
entities/items/{item_id}/labels	ラベル管理	CRUD
entities/items/{item_id}/descriptions	説明管理	CRUD

① CRUDとは

Create（作成）・**R**ead（読取）・**U**ppdate（更新）・**D**elete（削除）の略

18

Wikidata IDの調べ方

- Wikipedia記事 → サイドバー「ウィキデータ項目」→ URLの末尾
- または Action API で取得

Code

```
1 import requests
2
3 headers = {
4     "User-Agent": "IP3200-lecture/1.0 (https://rkskmt.github.io/ip3200/)"
5 }
6
7 params = {
8     "action": "query",
9     "prop": "pageprops",
10    "titles": "東京都",
11    "format": "json"
12 }
13 response = requests.get("https://ja.wikipedia.org/w/api.php",
14                          params=params, headers=headers)
15 data = response.json()
16
17 print(data["query"]["pages"])
```

pages は ページIDをキーにした辞書（例：**`{"774362": {"title": "東京都", "pageprops": {...}}}`**）

19

中身を取り出す

- キーがページID、値がページ情報の辞書
- **.items()** でキーと値のペアを順に取り出す

Code

```
1 for page_id, page_info in data["query"]["pages"].items():
2     wikidata_id = page_info["pageprops"]["wikibase_item"]
3     print(f"Wikidata ID: {wikidata_id}") # Q1490
```

20

実用サンプルコード

21

Action API：検索 + 詳細取得

Code

```
1 import requests
2
3 headers = {
4     "User-Agent": "IP3200-lecture/1.0 (https://rkskmt.github.io/ip3200/)"
5 }
6
7 # 検索：「桜」で5件
8 search_params = {
9     "action": "query",
10    "list": "search",
11    "srsearch": "桜",
12    "format": "json",
13    "srlimit": 5
14 }
15 r = requests.get("https://ja.wikipedia.org/w/api.php",
16                 params=search_params, headers=headers)
17 results = r.json()["query"]["search"]
18
19 for item in results:
20     print(f"- {item['title']}")
```

22

Action API：ページ詳細

- **formatversion: 2** を付けると、**pages** が「ページIDをキーにした辞書」ではなく **リスト** で返る（ループで素直に回せる）

Code

```
1 import requests
2
3 headers = {
4     "User-Agent": "IP3200-lecture/1.0 (https://rkskmt.github.io/ip3200/)"
5 }
6
7 detail_params = {
8     "action": "query",
9     "prop": "extracts|info|images",
10    "titles": "桜",
11    "format": "json",
12    "formatversion": 2, # JSONが扱いやすくなる
13    "exintro": True, # 導入部分のみ
14    "explaintext": True, # HTMLタグ除去
15    "redirects": True # リダイレクト自動追従
16 }
17 r = requests.get("https://ja.wikipedia.org/w/api.php",
18                 params=detail_params, headers=headers)
19 pages = r.json()["query"]["pages"]
20
21 for page in pages:
22     print(f"タイトル: {page['title']}")
23     print(f"概要: {page['extract'][:100]}...")
```

23

Action API：カテゴリ + ランダム

Code

```
1 import requests
2
3 headers = {
4     "User-Agent": "IP3200-lecture/1.0 (https://rkskmt.github.io/ip3200/)"
5 }
6
7 # カテゴリメンバー
8 cats_params = {
9     "action": "query",
10    "list": "categorymembers",
11    "cmtitle": "Category:桜",
12    "format": "json",
13    "cmlimit": 10
14 }
15 r = requests.get("https://ja.wikipedia.org/w/api.php",
16                 params=cats_params, headers=headers)
17 members = r.json()["query"]["categorymembers"]
18 for m in members:
19     print(f"- {m['title']}")
20
21 # ランダム記事
22 rnd_params = {
23     "action": "query", "list": "random",
24     "rnamespace": 0, "rnlimit": 3, "format": "json"
25 }
26 r = requests.get("https://ja.wikipedia.org/w/api.php",
27                 params=rnd_params, headers=headers)
28 for p in r.json()["query"]["random"]:
29     print(f"- {p['title']}")
```

24

REST API v1：概要取得

Code

```
1 import requests
2
3 headers = {
4     "User-Agent": "IP3200-lecture/1.0 (https://rkskmt.github.io/ip3200/)"
5 }
6
7 # 記事概要（タイトル、要約、画像URL）
8 response = requests.get(
9     "https://ja.wikipedia.org/api/rest_v1/page/summary/桜",
10    headers=headers
11 )
12 data = response.json()
13
14 print(f"タイトル: {data['title']}")
15 print(f"概要: {data['extract']}")
16 if "thumbnail" in data:
17     print(f"画像: {data['thumbnail']['source']}")
```

25

AQS：人気記事 + 編集統計

- この **items** は **1要素のリスト**（**[0]** で中身を取り出す） — per-articleの **items**（日ごとに1件）とは形が違う

```
Code
1 import requests
2
3 headers = {
4     "User-Agent": "IP3200-lecture/1.0 (https://rkskmt.github.io/ip3200/)"
5 }
6
7 # 人気記事ランキング
8 url = ("https://wikimedia.org/api/rest_v1/metrics/"
9       "pageviews/top/ja.wikipedia.org/all-access/2024/03/15")
10 r = requests.get(url, headers=headers)
11 articles = r.json()["items"][0]["articles"][:5]
12 for a in articles:
13     print(f"{a['rank']}. {a['article']} ({a['views'],} views)")
14
15 # 編集統計
16 url = ("https://wikimedia.org/api/rest_v1/metrics/"
17       "edits/aggregate/ja.wikipedia.org/all-editor-types/"
18       "all-page-types/monthly/20240101/20240331")
19 r = requests.get(url, headers=headers)
20 for item in r.json()["items"][0]["results"]:
21     print(f"{item['timestamp']}: {item['edits'],} edits")
```

26

統計データの活用

27

形を予想してから取りに行く

ページビューのグラフは、大きく2つの型に分かれる（**APIデータを可視化する**  で両方見た）。

型	形	きっかけ
スパイク型	1日だけの鋭い山	行事の当日・事件・訃報・テレビ放送（七夕の7/7）
波型	ゆるやかな季節の波	季節もの（富士山の夏の登山シーズン）

- 「桜」「クリスマス」「オリンピック」はどちらの型になりそう？
→ **予想してから**、次の演習のコードで確かめる
- 同じ記事の **日本語版 vs 英語版** で形を比べるのも面白い（地域で関心が違う）

28

演習

29

演習：好きな記事のPVを可視化



好きな**Wikipedia記事**の過去1年間の日別ページビューを取得し、折れ線グラフで描け（**最大アクセス日とその値**も表示）。

💡 ヒント

- AQS の **pageviews/per-article** を使う
- 記事名は日本語のままURLに入れてOK（requestsが自動エンコード）
- **APIでデータ取得の「七夕」のコード**  をベースに改造

▶ 解答例を見る

30

演習：カテゴリ内の記事を列挙



Action API で好きなカテゴリ に属する記事を10件取得し、各記事のタイトルを表示せよ。

```
Code
1 # カテゴリ例
2 # Category:日本の城
3 # Category:東京都の鉄道駅
4 # Category:日本のアニメ映画
```

▶ 解答例を見る

演習：REST API で概要取得



REST API v1 の `/page/summary/{title}` で好きな記事の タイトル・概要・サムネイ
ルURL を取得して表示せよ。

▶ 解答例を見る

関連リソース

リソース	URL
MediaWiki Action API	mediawiki.org/wiki/API:Main_page
Wikimedia REST API v1	en.wikipedia.org/api/rest_v1/
Analytics Query Service	wikitech.wikimedia.org
Wikidata REST API	wikidata.org/wiki/Wikidata:REST_API
User-Agent ポリシー	meta.wikimedia.org